

LIVRET PÉDAGOGIQUE · NIVEAU DÉBUTANT

React.js

Les fondamentaux pour bien démarrer
avec la bibliothèque JavaScript la plus utilisée au monde

Public : apprenant·e·s en formation développement web

Prérequis : HTML, CSS, JavaScript (bases ES6)

Version cible : React 19 · Vite · React Router 7

Durée indicative : 2 à 3 semaines d'apprentissage actif

Sommaire

1. 01. Bienvenue dans le monde de React	3
2. 02. Installer React : on démarre avec Vite	5
3. 03. JSX : écrire du HTML dans du JavaScript	7
4. 04. Le DOM virtuel : pourquoi React est rapide	9
5. 05. Composants et props : les briques de base	11
6. 06. Listes et rendu conditionnel	13
7. 07. Événements et state : rendre l'interface vivante	15
8. 08. Les formulaires en React	18
9. 09. useEffect : les effets de bord	20
10. 10. Faire communiquer les composants	23
11. 11. Aller plus loin avec les hooks	25
12. 12. Le routing avec React Router	28
13. 13. Styliser ses composants	30
14. 14. Déboguer et tester	32
15. 15. Bonnes pratiques et pièges à éviter	34
16. 16. Écosystème et suite du parcours	36
17. 17. Glossaire express	38
18. 18. Pour finir : ce qu'il faut retenir	39

Ce livret se lit dans l'ordre, mais tu peux aussi l'utiliser comme référence : chaque section est indépendante. Les exemples de code sont volontairement courts — recopie-les dans ton projet Vite pour les tester.

1. Bienvenue dans le monde de React

OBJECTIF Comprendre à quoi sert React, pourquoi il est partout, et ce qu'il faut savoir avant de commencer.

Si tu arrives ici, c'est que tu as déjà touché au HTML, au CSS et au JavaScript — et que tu as peut-être senti les limites de la méthode « je construis mes pages à la main ». React est la réponse moderne à cette limite : c'est une **bibliothèque JavaScript** qui permet de construire des **interfaces utilisateur (UI)** à base de petits blocs réutilisables appelés **composants**.

React est utilisé par des milliers d'entreprises : Meta (Facebook, Instagram), Airbnb, Netflix, Discord, Uber... Apprendre React, c'est apprendre une compétence réellement demandée sur le marché de l'emploi, particulièrement pour les métiers de **développeur·se web** et **fullstack**.

Pourquoi React ?

- **Déclaratif** : tu décris ce que tu veux afficher, React s'occupe du comment (mise à jour du DOM). Fini les manipulations manuelles du type `document.getElementById(...)`.
- **Composants** : ton interface est découpée en briques réutilisables, comme des LEGO®.
- **Écosystème énorme** : des milliers de bibliothèques, une documentation officielle excellente (react.dev) et une grande communauté pour t'aider.
- **Employabilité** : avec React dans ton CV, tu vises les ESN, agences web, éditeurs de logiciels...

Ce que tu dois déjà savoir (les prérequis)

React s'appuie sur du JavaScript moderne (ES6+). Avant de continuer, vérifie que ces notions ne te font pas peur :

Notion ES6+	Exemple minimal
Variables <code>const</code> / <code>let</code>	<code>const total = prix * quantite;</code>
Fonctions fléchées	<code>const double = (n) => n * 2;</code>
Template literals	<code>`Bonjour \${prenom} !`</code>
Destructuring	<code>const { nom, age } = user;</code>
Modules import / export	<code>import Bouton from './Bouton.jsx';</code>
Promesses / <code>async-await</code>	<code>const data = await fetch(url);</code>
Méthodes de tableau	<code>items.map(...)</code> , <code>items.filter(...)</code>

Si une de ces notions est floue, prends le temps de la revoir d'abord (MDN Web Docs est une excellente référence) : React deviendra beaucoup plus simple avec ces bases solides.

À RETENIR

- React = bibliothèque JavaScript pour construire des interfaces (pas un framework complet, contrairement à Angular ou Vue).
- Tout se construit avec des **composants** réutilisables.
- Les bases ES6 sont indispensables avant d'attaquer React.

2. Installer React : on démarre avec Vite

OBJECTIF Créer ton premier projet React proprement avec l'outil officiellement recommandé : Vite.

Historiquement, on créait un projet React avec **Create React App**. Cet outil n'est plus recommandé : l'équipe React recommande désormais **Vite**, plus rapide et plus simple.

Créer le projet

Ouvre un terminal dans le dossier où tu veux créer ton projet, puis tape :

```
npm create vite@latest mon-projet -- --template react
cd mon-projet
npm install
npm run dev
```

- `npm create vite@latest` télécharge le générateur officiel Vite ;
- `--template react` installe le modèle JavaScript de React (il existe aussi `react-ts` pour TypeScript) ;
- `npm install` installe les dépendances listées dans `package.json` ;
- `npm run dev` lance un serveur de développement local (généralement sur `http://localhost:5173`) avec **rechargement à chaud** : chaque sauvegarde met à jour la page instantanément.

STRUCTURE DU PROJET : LES FICHIERS IMPORTANTS

index.html : le point d'entrée HTML. Il contient une balise `<div id="root"></div>` vide : c'est là que React « injectera » toute ton application.

src/main.jsx : le fichier JavaScript qui démarre React et monte le composant racine dans cette div.

src/App.jsx : ton premier composant, la racine de ton interface.

package.json : la liste des dépendances et les scripts (`dev` , `build` , `preview`).

Regardons main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import App from './App.jsx'
import './index.css'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- `createRoot(...)` dit à React : « ce nœud du DOM est ma racine » ;
- `.render(<App />)` affiche le composant `App` ;

- `StrictMode` est un outil de développement : il double volontairement certains rendus et effets pour **repérer les bugs** (il ne fait rien en production).

PIÈGE CLASSIQUE

Tu n'écris **jamais** toi-même dans `index.html` le HTML de tes pages (sauf la div racine). Tout le contenu se construit dans les composants `.jsx`.

3. JSX : écrire du HTML dans du JavaScript

OBJECTIF Comprendre la syntaxe JSX, ses règles d'or et pourquoi elle change ta façon de coder.

Le JSX est une extension de syntaxe qui permet d'écrire du **HTML dans du JavaScript**. C'est lui qui rend React si agréable à lire : tu vois directement la structure de ton interface.

```
const element = <h1>Bonjour !</h1>;
```

Ce n'est pas du HTML, ni une chaîne de caractères : c'est du **sucre syntaxique** qui est transformé en appel JavaScript. React lit cette description et fabrique un arbre d'éléments (le **DOM virtuel**, cf. section 4).

Les 4 règles d'or du JSX

Règle	Exemple
1. Un seul élément racine par retour	<code>return <div>...</div></code> ou fragment <code><>...</></code>
2. Les attributs HTML deviennent camelCase	<code>className</code> , <code>htmlFor</code> , <code>onClick</code>
3. Du JavaScript entre accolades	<code><p>{maVariable}</p></code>
4. Toute balise doit être fermée	<code></code>

Règle 1 : un seul élément racine

Une fonction composant doit retourner **un seul** élément. Si tu as plusieurs éléments frères, enveloppe-les dans un fragment `<> ... </>` (qui n'ajoute aucune balise au DOM) :

```
function Carte() {
  return (
    <>
      <h2>Titre de la carte</h2>
      <p>Contenu de la carte.</p>
    </>
  );
}
```

Règle 3 : les accolades, ton meilleur ami

Tout ce qui est entre accolades est évalué comme du **JavaScript** :

```
const prenom = 'Aya';
const annee = new Date().getFullYear();

function Salutation() {
  return (
    <p>
      Bonjour {prenom}, nous sommes en {annee}.
      Total : {2 + 3} - {prenom.toUpperCase()}
    </p>
  );
}
```

PIÈGES CLASSIQUES

- On écrit `className` et non `class` (réservé au JavaScript).
- Les commentaires JSX s'écrivent `{/* commentaire */}`.
- On ne peut pas mettre un `if` directement dans le JSX (voir section 6 pour les alternatives).

À RETENIR

- JSX = HTML + JavaScript mélangés, transformé en appels React.
- Une fonction composant retourne un seul élément racine (fragment si besoin).
- Les accolades `{ }` permettent d'insérer n'importe quelle expression JavaScript.

4. Le DOM virtuel : pourquoi React est rapide

OBJECTIF Comprendre le mécanisme qui rend React performant et pourquoi on ne touche plus le DOM à la main.

Le problème du DOM classique

Le **DOM** (Document Object Model) est la représentation en mémoire de ta page HTML. Chaque modification du DOM déclenche des recalculs de style et de mise en page : les manipuler souvent et sans réfléchir rend une application lente.

Avec du JavaScript « classique », tu fais tout à la main :

```
// Approche impérative : on dit À la machine COMMENT faire
const list = document.getElementById('liste');
list.innerHTML = '';
users.forEach((user) => {
  const li = document.createElement('li');
  li.textContent = user.name;
  list.appendChild(li);
});
```

C'est verbeux, fragile, et c'est facile d'y introduire des bugs.

La solution React : le DOM virtuel

React garde en mémoire une **copie virtuelle légère** de ton interface (un objet JavaScript). Quand l'état change :

1. React reconstruit cette copie virtuelle (très rapide, ce n'est pas le vrai DOM) ;
2. Il **compare** l'ancienne copie et la nouvelle (étape appelée **reconciliation**) ;
3. Il applique au vrai DOM uniquement les **différences minimales**.

L'ANALOGIE DU TABLEAU BLANC

Imagine que tu modifies une liste de courses affichée sur un tableau blanc. Sans React, tu effaces tout et tu réécris chaque ligne. Avec React, tu ne modifies que la ligne qui a changé — et encore, seulement si elle a vraiment changé.

Conséquence pratique

Tu décris ton interface de façon **déclarative** : tu dis « voilà ce qui doit être affiché étant donné cet état », et React s'occupe du reste. Tu ne touches quasiment plus jamais au DOM directement (exception : `useRef`, section 11).

À RETENIR

- React compare des copies virtuelles et ne met à jour que ce qui change vraiment.
- On écrit du code déclaratif (« quoi afficher ») et non impératif (« comment le faire »).
- C'est ce mécanisme qui rend les grosses applications React fluides.

5. Composants et props : les briques de base

OBJECTIF Savoir créer un composant, le réutiliser et lui passer des données via les props.

Un composant, c'est une fonction

Un composant React est simplement une **fonction JavaScript qui retourne du JSX**.

```
function Welcome() {
  return <h1>Bienvenue sur Garage404 !</h1>;
}

export default Welcome;
```

Deux règles de nommage :

- le nom du composant commence par une **majuscule** (`Welcome` et non `welcome`) : c'est ce qui permet à React de distinguer un composant d'une balise HTML ;
- chaque fichier contient généralement un composant principal exporté (`export default`).

Les props : passer des données à un composant

Les **props** (pour « properties ») sont les paramètres d'un composant. Le parent passe des données, l'enfant les reçoit dans un objet :

```
function Welcome({ prenom, age }) {
  return (
    <p>
      Bonjour {prenom} ! Tu as {age} ans.
    </p>
  );
}

// Utilisation dans un autre composant :
function App() {
  return (
    <main>
      <Welcome prenom="Aya" age={23} />
      <Welcome prenom="Karim" age={28} />
    </main>
  );
}
```

Note : pour passer une valeur **JavaScript** (nombre, variable, objet...), on utilise des accolades : `age={23}`. Une chaîne simple peut être passée entre guillemets.

Les props sont en lecture seule

Un composant ne doit **jamais modifier ses props**. Elles arrivent du parent et appartiennent au parent. Si un composant doit changer quelque chose, c'est l'affaire du **state** (section 7) et du parent.

La composition : children

Tout composant reçoit une prop spéciale `children` : ce qui se trouve entre ses balises d'ouverture et de fermeture.

```
function Encadre({ children }) {  
  return <div className="cadre">{children}</div>;  
}  
  
function App() {  
  return (  
    <Encadre>  
      <h2>Un titre</h2>  
      <p>Du contenu imbriqué...</p>  
    </Encadre>  
  );  
}
```

BON RÉFLEXE

Quand un composant devient trop gros (plus de ~50-80 lignes, plusieurs responsabilités), découpe-le en sous-composants qui reçoivent des props. C'est la clé d'un code maintenable.

À RETENIR

- Composant = fonction JavaScript qui retourne du JSX (majuscule obligatoire).
- Props = données entrantes, en lecture seule, passées de parent à enfant (flux unidirectionnel).
- `children` permet de composer des composants comme des poupées russes.

6. Listes et rendu conditionnel

OBJECTIF Afficher des tableaux de données avec `map()` et contrôler ce qui s'affiche avec des conditions.

Afficher une liste avec `map()`

Pour afficher un tableau, on le transforme en tableau de JSX avec `map()` :

```
const apprenants = ['Aya', 'Karim', 'Léa', 'Sofiane'];

function Liste() {
  return (
    <ul>
      {apprenants.map((prenom) => (
        <li key={prenom}>{prenom}</li>
      ))}
    </ul>
  );
}
```

La prop `key` : obligatoire et stable

React utilise `key` pour identifier chaque élément de la liste et savoir **lequel a changé, été ajouté ou supprimé**. Sans `key` (ou avec des keys instables), React peut afficher des choses incorrectes lors des mises à jour.

- ☐ Utilise un **identifiant unique et stable** : `key={user.id}` ;
- ⚠ L'index du tableau (`key={index}`) est à éviter dès que la liste peut être réordonnée ou filtrée : c'est une source de bugs subtils ;
- ☐ Jamais de `Math.random()` ou de date comme `key` : elle changerait à chaque rendu.

```
const produits = [
  { id: 1, nom: 'Carte Arduino', prix: 24 },
  { id: 2, nom: 'Capteur IoT', prix: 9 },
];

function Catalogue() {
  return (
    <ul>
      {produits.map((p) => (
        <li key={p.id}>
          {p.nom} - {p.prix} €
        </li>
      ))}
    </ul>
  );
}
```

Le rendu conditionnel

On ne peut pas écrire `if` directement dans le JSX. Trois techniques :

```
// 1. L'opérateur && : afficher si la condition est vraie
{isConnecte && <p>Bienvenue !</p>}

// 2. L'opérateur ternaire : deux choix possibles
{isConnecte ? <p>Bienvenue !</p> : <button>Se connecter</button>}

// 3. Une variable calculée avant le return
let contenu;
if (isChargement) {
  contenu = <p>Chargement...</p>;
} else if (erreur) {
  contenu = <p className="erreur">{erreur}</p>;
} else {
  contenu = <Donnees />;
}
return <div>{contenu}</div>;
```

PIÈGE : LE 0 ET LES FAUSSES VALEURS

`{nbMessages && <p>Messages</p>}` affichera **0** à l'écran si `nbMessages` vaut 0 (car 0 est une valeur « falsy » qui est bien rendue). Préfère une comparaison explicite : `{nbMessages > 0 && ...}`.

À RETENIR

- Une liste = un `map()` qui retourne du JSX, chaque item avec une `key` unique et stable.
- Conditions en JSX : `&&`, ternaire, ou variable calculée.

7. Événements et state : rendre l'interface vivante

OBJECTIF Réagir aux actions de l'utilisateur (clics, saisies) et faire évoluer l'interface grâce à `useState`.

Les événements

En React, on attache un gestionnaire d'événement directement sur l'élément, avec une fonction :

```
function Bouton() {
  function handleClick() {
    console.log('Clic !');
  }

  return <button onClick={handleClick}>Clique-moi</button>;
}
```

On passe la **fonction** (sans l'appeler !) : `onClick={handleClick}` et non `onClick={handleClick()}`. Les événements courants : `onClick`, `onChange`, `onSubmit`, `onFocus`, `onMouseEnter` ...

Le state avec `useState`

Une prop est figée, mais une interface doit **réagir** (compteur, champ de saisie, onglet actif...). C'est le rôle du **state** : une donnée interne au composant dont React surveille les changements pour **re-rendre** le composant.

```
import { useState } from 'react';

function Compteur() {
  // [valeur actuelle, fonction pour la modifier]
  const [compte, setCompte] = useState(0);

  function augmenter() {
    setCompte(compte + 1);
  }

  return (
    <div>
      <p>Compteur : {compte}</p>
      <button onClick={augmenter}>+1</button>
    </div>
  );
}
```

Décomposons :

- `useState(0)` initialise le state à `0` ;
- `compte` est la valeur courante, lue au moment du rendu ;

- `setCompte(...)` est la **seule** façon de changer la valeur ; dès qu'on l'appelle, React re-rend le composant avec la nouvelle valeur.

Les 3 règles d'or du state

1. **Ne jamais modifier directement** : `compte++` ou `compte = compte + 1` ne déclenche aucun re-rendu. Toujours passer par le setter.
2. **Immuabilité** : pour un objet ou un tableau, on crée une nouvelle copie : `setUsers([...users, newUser])` et non `users.push(...)`.
3. **Mises à jour asynchrones** : le setter ne change pas la valeur instantanément. Si tu as besoin de la valeur précédente, utilise la forme fonctionnelle : `setCompte((c) => c + 1)`.

```
// ☐ Correct : copie puis modification
setPanier([...panier, article]);

// ☐ Incorrect : mutation directe, React ne verra rien
panier.push(article);
```

POURQUOI L'IMMUABILITÉ ?

React compare les anciennes et nouvelles valeurs pour décider quoi re-rendre. Si tu modifies un objet en place, l'ancienne et la nouvelle référence sont identiques : React ne détecte aucun changement. Créer une copie lui donne une nouvelle référence à comparer. C'est le cœur du modèle React.

Exemple complet : une saisie contrôlée

```
function Champ() {
  const [texte, setTexte] = useState('');

  return (
    <div>
      <input
        type="text"
        value={texte}
        onChange={(e) => setTexte(e.target.value)}
        placeholder="Tape ton prénom"
      />
      <p>Tu as écrit : {texte}</p>
    </div>
  );
}
```

Ici l'input est **contrôlé** : sa valeur affichée vient du state, et chaque frappe met à jour le state. L'interface et les données restent toujours synchronisées.

À RETENIR

- Le state = la mémoire du composant ; on le modifie uniquement avec le setter fourni par `useState`.
- Jamais de mutation directe : toujours créer une copie (spread `...`, etc.).
- On passe la fonction à l'événement, on ne l'appelle pas.

8. Les formulaires en React

OBJECTIF Maîtriser les formulaires contrôlés : saisie, soumission, validation de base.

En React, la meilleure pratique est de rendre les champs de formulaire **contrôlés** : chaque champ est lié à un morceau de state, et c'est le state qui fait foi.

Un formulaire complet

```
import { useState } from 'react';

function Inscription() {
  const [prenom, setPrenom] = useState('');
  const [email, setEmail] = useState('');
  const [erreur, setErreur] = useState('');

  function handleSubmit(e) {
    e.preventDefault(); // empêche le rechargement de la page

    if (!email.includes('@')) {
      setErreur('Adresse email invalide. ');
      return;
    }

    console.log('Envoi des données :', { prenom, email });
    setPrenom('');
    setEmail('');
    setErreur('');
  }

  return (
    <form onSubmit={handleSubmit}>
      <label>
        Prénom
        <input
          value={prenom}
          onChange={(e) => setPrenom(e.target.value)}
        />
      </label>
      <label>
        Email
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
        />
      </label>
      {erreur && <p className="erreur">{erreur}</p>}
      <button type="submit">S'inscrire</button>
    </form>
  );
}
```

Les points clés

- `e.preventDefault()` : sans lui, le formulaire recharge la page et on perd tout ;
- `onSubmit` se déclenche au clic sur le bouton `type="submit"` ou à la touche Entrée ;
- la validation se fait dans le gestionnaire, et les erreurs s'affichent conditionnellement ;
- on peut regrouper plusieurs champs dans un seul objet state si le formulaire grossit.

```
const [form, setForm] = useState({ prenom: '', email: '' });

function handleChange(e) {
  const { name, value } = e.target;
  setForm((ancien) => ({ ...ancien, [name]: value }));
}

// <input name="prenom" value={form.prenom} onChange={handleChange} />
// <input name="email" value={form.email} onChange={handleChange} />
```

FORMULAIRES NON CONTRÔLÉS

Il existe aussi des champs « non contrôlés » (avec `useRef`), où React ne suit pas la valeur : utiles pour des cas très simples. Mais pour 95 % des formulaires, le mode contrôlé est plus prévisible et plus facile à valider.

À RETENIR

- Champ contrôlé = `value` + `onChange` reliés à un state.
- Toujours `e.preventDefault()` dans `onSubmit`.

9. useEffect : les effets de bord

OBJECTIF Gérer ce qui sort du rendu : appels réseau, timers, écouteurs, et savoir nettoyer derrière soi.

Un composant React doit rester **pur** : son rendu ne doit pas déclencher d'appels réseau, de timers ou de manipulation du DOM. Toutes ces opérations sont des **effets de bord** qui se placent dans `useEffect`.

```
import { useEffect } from 'react';

function Titre() {
  useEffect(() => {
    document.title = 'Ma super app';
  }, []); // tableau de dépendances

  return <p>Regarde l'onglet du navigateur !</p>;
}
```

Le tableau de dépendances : le vrai sujet

Dépendances	Quand l'effet s'exécute
<code>useEffect(fn)</code>	Après chaque rendu (rarement voulu)
<code>useEffect(fn, [])</code>	Une seule fois, après le montage
<code>useEffect(fn, [a, b])</code>	Au montage, puis à chaque fois que <code>a</code> ou <code>b</code> change

Charger des données depuis une API

```
import { useEffect, useState } from 'react';

function Apprenants() {
  const [apprenants, setApprenants] = useState([]);
  const [chargement, setChargement] = useState(true);

  useEffect(() => {
    const controller = new AbortController();

    async function charger() {
      try {
        const reponse = await fetch('/api/apprenants', {
          signal: controller.signal,
        });
        const donnees = await reponse.json();
        setApprenants(donnees);
      } catch (erreur) {
        console.error('Erreur de chargement :', erreur);
      } finally {
        setChargement(false);
      }
    }

    charger();

    // Nettoyage : annule la requête si le composant est démonté
    return () => controller.abort();
  }, []);

  if (chargement) return <p>Chargement...</p>;
  return (
    <ul>
      {apprenants.map((a) => (
        <li key={a.id}>{a.nom}</li>
      ))}
    </ul>
  );
}
```

Le nettoyage (cleanup)

Si ton effet met en place quelque chose (timer, écouteur d'événement, abonnement, requête), tu dois le **nettoyer** quand le composant disparaît ou quand l'effet se relance. La fonction retournée par l'effet est le cleanup :

```
useEffect(() => {
  const id = setInterval(() => {
    console.log('tick');
  }, 1000);

  return () => clearInterval(id); // nettoyage
}, []);
```

LES PIÈGES À ÉVITER

- **Dépendances incomplètes** : si ton effet utilise une variable, elle doit figurer dans le tableau. L'ESLint ([react-hooks/exhaustive-deps](#)) te le signale — écoute-le.
- **setState en boucle** : un effet qui met à jour le state sans dépendances stables provoque un rendu infini. Pense toujours à ce que ton effet dépend de.
- **Effet inutile** : beaucoup d'effets peuvent être supprimés en calculant pendant le rendu ou en gérant dans l'événement. Avant d'écrire un `useEffect`, demande-toi s'il est vraiment nécessaire.

À RETENIR

- `useEffect` = « après le rendu, fais ça » (fetch, timers, écouteurs...).
- Le tableau de dépendances contrôle la fréquence d'exécution ; `[]` = une fois au montage.
- Toujours nettoyer ce qu'on a mis en place (retour d'une fonction de cleanup).

10. Faire communiquer les composants

OBJECTIF Transmettre des données et des actions entre composants : props, lifting state up, composition, contexte.

Le flux de données : de haut en bas

En React, les données circulent **du parent vers l'enfant** via les props. Un enfant peut aussi recevoir une **fonction** en prop pour « remonter » une information au parent :

```
function BoutonFavori({ onToggle, estFavori }) {
  return (
    <button onClick={onToggle}>
      {estFavori ? '★ Retirer' : '☆ Ajouter'}
    </button>
  );
}

function Article({ titre }) {
  const [favori, setFavori] = useState(false);
  return (
    <article>
      <h3>{titre}</h3>
      <BoutonFavori
        estFavori={favori}
        onToggle={() => setFavori(!favori)}
      />
    </article>
  );
}
```

Lifting state up : remonter le state

Quand **plusieurs composants frères** doivent partager la même donnée, on place le state dans leur **parent commun** et on le distribue par props :

```
function App() {
  // Le state vit ici, dans le parent commun
  const [selection, setSelection] = useState('');

  return (
    <div>
      <SelecteurCouleur valeur={selection} onChange={setSelection} />
      <Apercu couleur={selection} />
    </div>
  );
}
```

Le parent détient la vérité (single source of truth), les enfants deviennent simples et réutilisables. C'est LA technique à maîtriser avant le contexte.

Le contexte : éviter le « props drilling »

Quand une donnée doit traverser beaucoup de niveaux (thème, utilisateur connecté, langue...), les passer comme props devient pénible : c'est le **props drilling**.

```
import { createContext, useContext, useState } from 'react';

// 1. Créer le contexte
const ThemeContext = createContext('clair');

// 2. Le fournisseur enveloppe l'application
function App() {
  const [theme, setTheme] = useState('clair');

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Page />
    </ThemeContext.Provider>
  );
}

// 3. N'importe quel descendant consomme sans props
function BoutonTheme() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme(theme === 'clair' ? 'sombre' : 'clair')}>
      Thème actuel : {theme}
    </button>
  );
}
```

QUAND (NE PAS) UTILISER LE CONTEXTE

Le contexte est pratique mais il rend les composants **moins réutilisables** (ils dépendent d'un fournisseur invisible). Règle simple : commence par les props, passe au lifting state up si besoin, et n'introduis le contexte que pour des données réellement globales (thème, session utilisateur, langue). Pour de gros états globaux, regarde Zustand ou Redux (section 16).

À RETENIR

- Props : de parent à enfant ; fonctions en props : de l'enfant vers le parent.
- Plusieurs frères partagent une donnée → on remonte le state au parent commun.
- Le contexte évite le props drilling, mais avec parcimonie.

11. Aller plus loin avec les hooks

OBJECTIF Découvrir `useReducer`, `useRef`, `useMemo`, `useCallback` et apprendre à créer ses propres hooks.

Les règles des hooks (à connaître par cœur)

1. Les hooks s'appellent uniquement **au niveau racine** du composant (pas dans des `if`, boucles ou fonctions imbriquées) ;
2. Les hooks s'appellent uniquement dans des **composants React** ou d'autres hooks (pas dans des fonctions classiques).

Le linter `eslint-plugin-react-hooks` vérifie ces règles pour toi.

useReducer : quand le state devient complexe

Quand un composant accumule plusieurs `useState` qui évoluent ensemble, on peut regrouper la logique dans un **réducteur** (une fonction qui reçoit l'état et une action, et retourne le nouvel état) :

```
import { useReducer } from 'react';

function reducer(etat, action) {
  switch (action.type) {
    case 'incrémenter':
      return { ...etat, compte: etat.compte + 1 };
    case 'décrémenter':
      return { ...etat, compte: etat.compte - 1 };
    case 'réinitialiser':
      return { compte: 0 };
    default:
      return etat;
  }
}

function Compteur() {
  const [etat, dispatch] = useReducer(reducer, { compte: 0 });

  return (
    <div>
      <p>Compte : {etat.compte}</p>
      <button onClick={() => dispatch({ type: 'incrémenter' })}>+1</button>
      <button onClick={() => dispatch({ type: 'décrémenter' })}>-1</button>
      <button onClick={() => dispatch({ type: 'réinitialiser' })}>Reset</button>
    </div>
  );
}
```

Utile aussi pour remplacer le contexte + state par context + reducer (le pattern de beaucoup d'applications réelles). Mais ne force pas : pour 80 % des cas, `useState` suffit.

useRef : une valeur qui ne déclenche pas de re-rendu

`useRef` sert à deux choses : conserver une valeur qui ne doit pas provoquer de rendu, et accéder directement à un élément du DOM :

```
import { useRef } from 'react';

function FocusAuto() {
  const champRef = useRef(null);

  function focusChamp() {
    champRef.current.focus();
  }

  return (
    <div>
      <input ref={champRef} type="text" />
      <button onClick={focusChamp}>Focus !</button>
    </div>
  );
}
```

REF VS STATE

Modifier une ref ne re-rend PAS le composant ; modifier un state le re-rend. Pour ce qui doit apparaître à l'écran → state. Pour ce qui est « invisible » (compteur d'itérations, pointeur DOM) → ref.

useMemo et useCallback : optimiser avec précaution

```
import { useMemo, useCallback } from 'react';

// useMemo : mémorise le RÉSULTAT d'un calcul coûteux
const total = useMemo(
  () => panier.reduce((somme, p) => somme + p.prix * p.quantite, 0),
  [panier], // recalculé seulement si panier change
);

// useCallback : mémorise la FONCTION elle-même
const ajouter = useCallback((produit) => {
  setPanier((ancien) => [...ancien, produit]);
}, []);
```

⚠ Ce ne sont pas des optimisations gratuites : elles ont un coût mémoire et de code. La règle : **profile d'abord, optimise ensuite**. Elles deviennent utiles quand un calcul est lourd ou quand des fonctions instables cassent la mémorisation de composants enfants coûteux (`React.memo`).

Créer ses propres hooks

Un custom hook est une fonction qui commence par `use` et qui utilise d'autres hooks. C'est le meilleur moyen de **réutiliser de la logique** entre composants :

```
import { useEffect, useState } from 'react';

function useLocalStorage(cle, valeurInitiale) {
  const [valeur, setValeur] = useState(() => {
    const stockee = localStorage.getItem(cle);
    return stockee !== null ? JSON.parse(stockee) : valeurInitiale;
  });

  useEffect(() => {
    localStorage.setItem(cle, JSON.stringify(valeur));
  }, [cle, valeur]);

  return [valeur, setValeur];
}

// Utilisation : const [prenom, setPrenom] = useLocalStorage('prenom', '');
```

À RETENIR

- Hooks : toujours au niveau racine, toujours dans un composant ou un custom hook.
- `useReducer` pour un state complexe ; `useRef` pour le DOM ou les valeurs sans rendu.
- `useMemo` / `useCallback` : seulement quand c'est utile.
- Les custom hooks commencent par `use` et encapsulent de la logique réutilisable.

12. Le routing avec React Router

OBJECTIF Créer une application multi-pages côté client : routes, liens, paramètres d'URL.

Une application React est une **SPA** (Single Page Application) : un seul fichier HTML chargé une fois. Pour simuler plusieurs pages sans recharger le navigateur, on utilise un routeur. Le standard est **React Router** (version 6+/7).

Installation et mise en place

```
npm install react-router-dom
```

Déclarer les routes

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const routeur = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { index: true, element: <Accueil /> },
      { path: 'apprenants', element: <Apprenants /> },
      { path: 'apprenants/:id', element: <FicheApprenant /> },
      { path: '*', element: <PageIntrouvable /> },
    ],
  },
]);

createRoot(document.getElementById('root')).render(
  <RouterProvider router={routeur} />,
);
```

Les liens et les paramètres

```
import { Link, NavLink, Outlet, useParams } from 'react-router-dom';

function Layout() {
  return (
    <div>
      <nav>
        <NavLink to="/">Accueil</NavLink>
        <NavLink to="/apprenants">Apprenants</NavLink>
      </nav>
      { /* C'est ici que s'affiche la route enfant */ }
      <Outlet />
    </div>
  );
}

function FicheApprenant() {
  const { id } = useParams(); // lit l'URL /apprenants/42
  return <p>Fiche de l'apprenant n° {id}</p>;
}
```

- **Link** : navigation sans rechargement (toujours utiliser Link plutôt que `<a href>` pour les liens internes) ;
- **NavLink** : comme Link, avec une classe **active** automatique sur la route courante ;
- **Outlet** : l'emplacement où les routes enfants s'affichent (layout partagé) ;
- **useParams** : lit les paramètres dynamiques de l'URL ;
- **path: '*'** : la route « 404 ».

VRAIE NAVIGATION VS SPA

Dans une SPA, la navigation ne recharge pas la page : c'est le routeur qui met à jour l'URL et le contenu. Résultat : des transitions instantanées, dignes d'une application native.

13. Styliser ses composants

OBJECTIF Organiser le CSS d'une application React : fichiers CSS, CSS Modules, et panorama des autres options.

React n'impose aucun outil de style : tu choisis. Voici les options du plus simple au plus avancé.

1. Fichier CSS global (pour démarrer)

```
import './App.css';

function App() {
  return <button className="bouton-primaire">Cliquez</button>;
}
```

Simple, mais tous les sélecteurs sont **globaux** : attention aux collisions de noms de classes.

2. CSS Modules (recommandé pour la plupart des projets)

Le fichier CSS est scopé au composant : les noms de classes sont automatiquement rendus uniques.

```
/* Bouton.module.css */
.bouton {
  background: #146ef5;
  color: white;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
}
```

```
import styles from './Bouton.module.css';

function Bouton() {
  return <button className={styles.bouton}>Cliquez</button>;
}
```

3. Les autres options

Option	Idée	Quand l'utiliser
Tailwind CSS	Classes utilitaires directement en JSX	Projets rapides, design systems
CSS-in-JS (styled-components, Emotion)	Styles écrits en JavaScript	Thème dynamique, composants de bibliothèque
Frameworks (Bootstrap, MUI, Chakra...)	Composants prêts à l'emploi	Gagner du temps sur des interfaces standard

RÈGLE D'OR

Peu importe l'outil, garde une structure de classes claire et cohérente (ex. BEM), et pense **mobile-first** avec les media queries.

14. Déboguer et tester

OBJECTIF Outiller son environnement : React DevTools, StrictMode, et les bases du test de composants.

React DevTools

Extension navigateur (Chrome, Firefox, Edge) indispensable. Elle ajoute deux onglets dans les outils développeur :

- **Components** : inspecte l'arbre de composants, les props et le state en direct ;
- **Profiler** : mesure les performances de rendu (à utiliser quand l'app rame).

StrictMode

Déjà vu à la section 2 : en développement, il double les rendus et les effets pour révéler les bugs (par exemple un effet sans cleanup qui fuit). **Laisse-le activé** pendant le développement.

Tester ses composants

L'outillage standard du monde React : **Vitest** (runner de tests) + **React Testing Library** (interactions avec les composants, comme le ferait un utilisateur).

```
npm install -D vitest @testing-library/react @testing-library/jest-dom @testing-library/user-event
```

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import Compteur from './Compteur';

test('le compteur s'incrémente au clic', async () => {
  const user = userEvent.setup();
  render(<Compteur />);

  await user.click(screen.getByRole('button', { name: '+1' }));

  expect(screen.getByText('Compteur : 1')).toBeInTheDocument();
});
```

On teste le **comportement visible** (ce qu'un utilisateur voit et fait), pas les détails d'implémentation. Commence par tester les parties critiques : logique métier et interactions importantes.

À RETENIR

- React DevTools = ton meilleur allié pour comprendre ce que fait ton app.
- StrictMode aide à détecter les bugs tôt : on le garde.
- Testing Library teste comme un utilisateur : par le rôle et le texte visibles.

15. Bonnes pratiques et pièges à éviter

OBJECTIF Adopter les réflexes des développeurs React professionnels et éviter les erreurs les plus courantes.

Les bonnes pratiques

- **Composants purs et prévisibles** : un composant rendu deux fois avec les mêmes props doit produire le même résultat. Pas de logique « sale » pendant le rendu.
- **Découper intelligemment** : un composant = une responsabilité. Si un fichier devient illisible, extrais des sous-composants.
- **Nommer clairement** : composants en `PascalCase`, gestionnaires d'événements en `handleXxx`, hooks custom en `useXxx`.
- **Accessibilité (a11y)** : `alt` sur les images, labels sur les champs, boutons réels (`<button>` et non des `<div>` cliquables), navigation au clavier.
- **Séparer les données de l'affichage** : la logique métier (calculs, filtres) vit dans des fonctions et hooks dédiés, pas dans le JSX.
- **Écouter le linter** : l'ESLint officiel de React signale la plupart des pièges (hooks, dépendances d'effets, clés de listes).

Les pièges les plus courants

Piège	Pourquoi c'est grave	La bonne habitude
Mutiter un state (push, obj.x = ...)	React ne détecte pas le changement, UI désynchronisée	Toujours une copie : <code>[...arr]</code> , <code>{...obj}</code>
<code>key={index}</code> sur des listes dynamiques	État d'items mélangés lors des réordonnancements	Un <code>id</code> stable
Appeler le setter pendant le rendu	Rendu infini ou comportement imprévisible	Setter dans les événements ou les effets
Oublier les dépendances d'effet	Données obsolètes, requêtes en boucle	Inclure les variables utilisées, écouter ESLint
Énormes composants « fourre-tout »	Code impossible à maintenir et à tester	Découper + composer

Performance : les bases

- Réduire le nombre de re-rendus : state au bon endroit (le plus bas possible dans l'arbre) ;
- `useMemo` / `useCallback` et `React.memo` uniquement sur des cas identifiés (le Profiler de DevTools te le dira) ;
- `React.lazy` + `Suspense` pour charger les grosses pages seulement quand on en a besoin (code splitting) ;

- Images optimisées (formats modernes, dimensions correctes) — c'est souvent là que se joue la vraie perf.

LA RÈGLE N°1

Lis les erreurs du navigateur et de la console : 90 % des problèmes React y sont expliqués en toutes lettres, avec la solution. Un réflexe de pro : lire, comprendre, corriger.

16. Écosystème et suite du parcours

OBJECTIF Savoir où aller après ce livret : outils complémentaires, frameworks, ressources officielles.

Les briques complémentaires (à découvrir progressivement)

Besoin	Outil standard	Quand
Typage	TypeScript	Dès que le projet grossit (la majorité des offres d'emploi le demandent)
État global	Zustand (simple) ou Redux Toolkit (structuré)	État partagé complexe au-delà du contexte
Données serveur	TanStack Query	Cache, rechargement, synchronisation avec une API
Formulaires	React Hook Form + Zod	Formulaires longs avec validation
Framework complet	Next.js	SEO, rendu serveur (SSR), routeurs fichier, plein emploi
Mobile	React Native	Applications iOS/Android avec les mêmes compétences

Ressources officielles (gratuites)

- **react.dev** — la documentation officielle, entièrement refaite. Commence par « Learn React » et fais les exercices interactifs ;
- **react.dev/learn/thinking-in-react** — la démarche de conception à connaître ;
- **vite.dev** — l'outil de build ;
- **reactrouter.com** — la doc de React Router ;
- **developer.mozilla.org (MDN)** — pour consolider JavaScript, HTML, CSS ;
- **ESLint + plugin React** — configure-le dès le premier projet, il devient ton professeur.

Un parcours d'apprentissage réaliste

1. Maîtrise les sections 1 à 9 de ce livret en construisant de petits composants (compteur, liste de tâches, recherche filtrée) ;
2. Ajoute la communication entre composants (section 10) et le routing (section 12) sur un mini-projet ;
3. Construis un projet complet de A à Z : formulaire + liste + détail (ex. gestionnaire de tâches avec API locale) ;
4. Introduis TypeScript sur ton prochain projet ;
5. Découvre Next.js pour les applications « réelles » (SSR, SEO).

LE CONSEIL DU FORMATEUR

Ne lis pas ce livret passivement : ouvre Vite, recopie chaque exemple, modifie-le, casse-le, répare-le. La compétence React s'acquiert en **faisant** — c'est exactement la philosophie « learning by doing » de Garage404.

17. Glossaire express

OBJECTIF Les 20 mots à connaître pour parler React avec les développeurs.

Terme	Définition courte
Composant	Fonction qui retourne du JSX ; brique réutilisable de l'UI.
JSX	Syntaxe qui permet d'écrire du HTML dans du JavaScript.
Props	Données passées d'un parent à un enfant, en lecture seule.
State	Donnée interne d'un composant ; sa modification déclenche un re-rendu.
Hook	Fonction React (useState, useEffect...) qui « branche » un composant sur des fonctionnalités.
Rendu (render)	Le fait de produire l'affichage d'un composant à partir de son état.
Re-rendu	Nouvelle exécution de la fonction composant suite à un changement d'état ou de props.
DOM virtuel	Représentation en mémoire de l'UI, comparée pour ne mettre à jour que le strict nécessaire.
Réconciliation	Algorithme de comparaison entre l'ancien et le nouveau rendu virtuel.
key	Identifiant stable requis pour chaque élément d'une liste rendue.
Événement	Interaction utilisateur (clic, saisie...) gérée par des fonctions comme onClick.
Effet de bord	Opération hors rendu (fetch, timer...) à placer dans useEffect.
Composant contrôlé	Champ de formulaire dont la valeur est pilotée par le state.
Lifting state up	Remonter un state vers le parent commun pour le partager entre enfants.
Props drilling	Faire traverser des props à travers beaucoup de niveaux de composants.
Contexte	Mécanisme pour rendre une valeur disponible à tout un arbre de composants.
SPA	Single Page Application : une page HTML, navigation sans rechargement.
Routeur	Bibliothèque (React Router) qui gère les « pages » d'une SPA.
Rechargement à chaud (HMR)	Mise à jour instantanée de la page à chaque sauvegarde (Vite).
StrictMode	Mode de développement qui double les rendus pour révéler les bugs.
Cleanup	Fonction de nettoyage retournée par un effet (timers, abonnements, requêtes).

18. Pour finir : ce qu'il faut retenir

OBJECTIF La synthèse en une page : les 10 réflexes qui font de toi un développeur React.

1. React, c'est des **composants** : des fonctions qui retournent du JSX.
2. Les **props** descendent (parent → enfant), les **fonctions** remontent (enfant → parent).
3. Le **state** est la mémoire du composant : on le modifie uniquement avec le setter, jamais par mutation.
4. On ne touche pas au DOM à la main : on décrit, React s'occupe des différences.
5. Les listes veulent des `key` stables ; les conditions s'écrivent avec `&&` ou le ternaire.
6. Les effets de bord vont dans `useEffect`, avec dépendances et cleanup.
7. Donnée partagée entre frères → on **remonte le state** ; donnée globale → contexte.
8. Le linter et les messages d'erreur de la console sont tes professeurs : lis-les.
9. Optimise seulement quand c'est mesuré ; la lisibilité prime.
10. Apprends en **construisant** : un projet vaut mieux que dix cours.

Bravo d'être arrivé au bout ! Ce livret est un point de départ : la vraie progression se fera projet après projet, erreur après erreur. N'hésite jamais à poser des questions — à ton formateur, à la communauté, et à tes pairs. C'est comme ça qu'on devient développeur.

— L'équipe pédagogique Garage404